

Ordenação Polimórfica em Software

Padrões de Comparação e Desacoplamento de Algoritmos em Orientação a Objetos

ABORDAGEM 1 Java: Interface Nativa Comparable<T>

Implementação onde o critério de comparação é encapsulado diretamente dentro da hierarquia da classe base abstrata. O algoritmo nativo de ordenação aciona a comparação polimórfica sem conhecer as subclasses concretas.

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

// Classe abstrata base
abstract class ItemCatalogo implements Comparable<ItemCatalogo> {
    private String titulo;
    private double preco;

    public ItemCatalogo(String titulo, double preco) {
        this.titulo = titulo;
        this.preco = preco;
    }

    public String getTitulo() { return titulo; }
    public double getPreco() { return preco; }

    // Método abstrato para calcular o valor relativo/relevância
    public abstract double calcularPrioridade();

    // Critério de ordenação polimórfico (por prioridade descendente)
    @Override
    public int compareTo(ItemCatalogo outro) {
        return Double.compare(outro.calcularPrioridade(), this.calcularPrioridade());
    }
}

// Subclasse 1: Livro
class Livro extends ItemCatalogo {
    private int paginas;

    public Livro(String titulo, double preco, int paginas) {
        super(titulo, preco);
        this.paginas = paginas;
    }

    @Override
    public double calcularPrioridade() {
        return paginas / getPreco();
    }
}

// Subclasse 2: Jogo
class Jogo extends ItemCatalogo {
    private int metacritic;

    public Jogo(String titulo, double preco, int metacritic) {
        super(titulo, preco);
        this.metacritic = metacritic;
    }

    @Override
    public double calcularPrioridade() {
        return metacritic;
    }
}
```

ABORDAGEM 2 Java: Padrão Strategy com Comparator<T>

Utilização do padrão Strategy para permitir múltiplos critérios de ordenação dinamicamente selecionáveis, aproveitando o polimorfismo das classes de domínio através de referências de método.

```
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

abstract class Veiculo {
    private String modelo;
    private int ano;

    public Veiculo(String modelo, int ano) {
        this.modelo = modelo;
        this.ano = ano;
    }

    public String getModelo() { return modelo; }
    public int getAno() { return ano; }
    public abstract double getCustoManutencao();
}

class Carro extends Veiculo {
    private double quilometragem;

    public Carro(String modelo, int ano, double quilometragem) {
        super(modelo, ano);
        this.kilometragem = quilometragem;
    }

    @Override
    public double getCustoManutencao() {
        return quilometragem * 0.15;
    }
}

class Moto extends Veiculo {
    private int cilindradas;

    public Moto(String modelo, int ano, int cilindradas) {
        super(modelo, ano);
        this.cilindradas = cilindradas;
    }

    @Override
    public double getCustoManutencao() {
        return cilindradas * 1.2;
    }
}

public class ExemploComparator {
    public static void main(String[] args) {
        List<Veiculo> frota = Arrays.asList(
            new Carro("Sedan", 2020, 45000),
            new Moto("Esportiva", 2022, 600),
            new Carro("Hatch", 2018, 80000)
        );

        // Comparator polimórfico baseado no retorno do método abstrato
        Comparator<Veiculo> porCusto = Comparator.comparingDouble(Veiculo::getCustoManutencao);

        frota.sort(porCusto);

        frota.forEach(v -> System.out.println(v.getModelo() + " | Custo: R$ " +
            v.getCustoManutencao()));
    }
}
```

ABORDAGEM 3 TypeScript: QuickSort Polimórfico Genérico

Implementação de um algoritmo de ordenação customizado (QuickSort) genérico que interage apenas com a interface pública da classe base abstrata.

```
abstract class EntidadeSistema {
  constructor(public readonly id: string) {}

  // Método polimórfico abstrato
  abstract ehMenorQue(outra: EntidadeSistema): boolean;
}

class Transacao extends EntidadeSistema {
  constructor(id: string, public valor: number) {
    super(id);
  }

  ehMenorQue(outra: EntidadeSistema): boolean {
    if (outra instanceof Transacao) {
      return this.valor < outra.valor;
    }
    return this.id < outra.id;
  }
}

// Algoritmo de Ordenação Genérico e Polimórfico
function quickSortPolimorfico<T extends EntidadeSistema>(arr: T[]): T[] {
  if (arr.length <= 1) return arr;

  const pivo = arr[arr.length - 1];
  const menores: T[] = [];
  const maiores: T[] = [];

  for (let i = 0; i < arr.length - 1; i++) {
    if (arr[i].ehMenorQue(pivo)) {
      menores.push(arr[i]);
    } else {
      maiores.push(arr[i]);
    }
  }

  return [...quickSortPolimorfico(menores), pivo, ...quickSortPolimorfico(maiores)];
}
```

Vantagens da Ordenação Polimórfica

Princípio do Aberto/Fechado (OCP)

Novas subclasses podem ser adicionadas sem alterar nenhuma linha do algoritmo de ordenação existente.

Desacoplamento Forte

A rotina de ordenação conhece apenas os métodos de comparação da interface base, garantindo maior reuso de código.